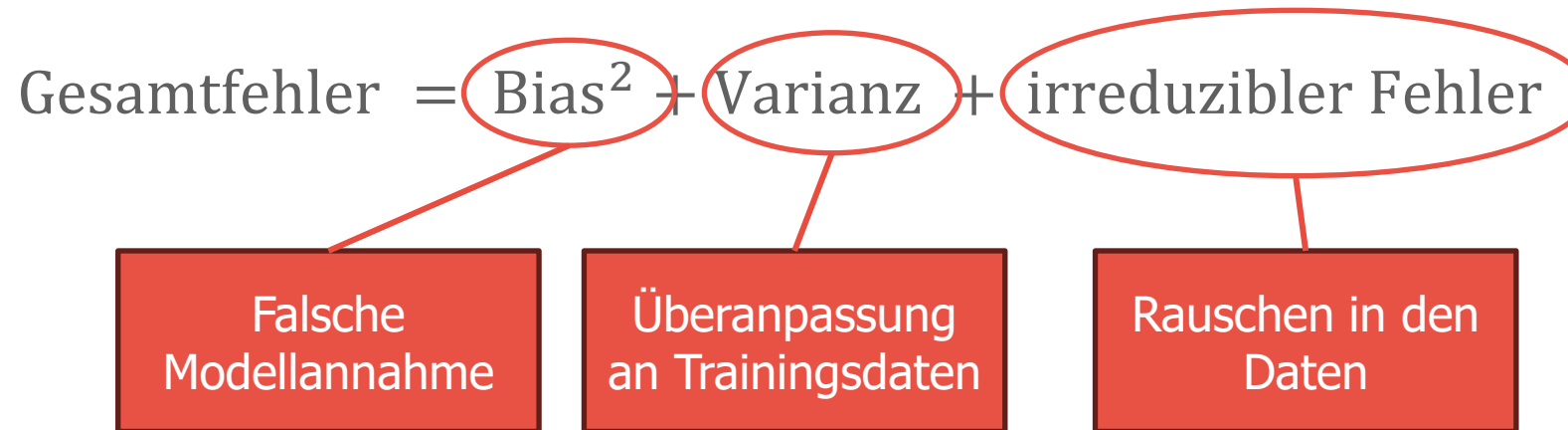


# Regularisierung



**FH KUFSTEIN TIROL**  
University of Applied Sciences

- » Der Fehler eines Modells  $\hat{f}$  kann wie folgt in drei Teile zerlegt werden:



- » Gegen den irreduziblen Fehler können wir nichts unternehmen.

- » **Definition:** Methoden um die Komplexität des Modells zu steuern.
- » **Ziel:** Generalisierbarkeit des Modells zu verbessern
- » **Methoden:**
  - Harte Regularisierung (Einschränkung)
  - Weiche Regularisierung (Strafterm)
  - Output Regularisierung
  - Early Stopping
  - Regularisierung durch Design
  - Dropout
  - Label Smoothing
  - ...
- » Oft werden die Namen von Algorithmen verwendet, statt den Namen der Regularisierungsmethode

- » Wir wollen ein Modell  $f_\theta$  mit Gewichten  $\theta \in \mathbb{R}^d$  finden. Hier bezeichnet  $d$  die Anzahl der Parameter des Modells.
- » Ohne Regularisierung würden wir die Lossfunktion

$$\mathcal{L}(\theta; X, Y) = \frac{1}{N} \sum_{i=1}^N \mathcal{D}(f_\theta(x_i), y_i)$$

für gegebene Daten  $(X, Y) = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$

- » Häufig ist  $\mathcal{D}(f_\theta(x_i), y_i) = \|f_\theta(x_i) - y_i\|^2$
- » Oft werden gewisse Bedingungen an  $f_\theta$  bzw.  $\theta$  gestellt um die Varianz zu verringern und Generalisierbarkeit positiv zu beeinflussen.

# Prior Knowledge (Vorwissen)

- » Vorwissen über das Modell oder den Output hilft uns die passende Regularisierungsmethode zu wählen
- » Wir können mit Prior-Knowledge auch eine neue Regularisierungsmethode designen
- » Probabilistische Sichtweise werden wir uns später kurz anschauen

- » Bei der harten Regularisierung wird die Modellklasse strikt eingeschränkt.
- » Am häufigsten wird dies mit einer Gleichung oder Ungleichung gemacht, es kann aber generell eine beliebige Aussage sein.
- » Die Lossfunktion wird dann mit einer Bedingung optimiert:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

mit  $\theta \in C \subset \mathbb{R}^d$ .

- » Bei der weichen Regularisierung (soft regularization oder variationelle Regularisierung) wird die Bedingung in ein Funktional umgewandelt und zum loss hinzugefügt.
- » Ein Funktional ist eine Funktion  $\mathcal{R}: \mathbb{R}^d \rightarrow [0, \infty)$
- » Man minimiert also:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N ||f_{\theta}(x_i) - y_i||^2 + \lambda \mathcal{R}(\theta)$$

- » Man nennt die Zahl  $\lambda \geq 0$  den Regularisierungsparameter und er gibt an wie stark die Bestrafung in der Optimierung einfließt.

- » Bei der Output Regularisierung werden nicht die Parameter des Modells direkt eingeschränkt/bestraft, sondern der Output des Modells.
- » Liefert eine Möglichkeit gewollte Eigenschaften des Outputs zu erreichen.
- » Wird im machine learning Kontext häufig nicht erwähnt.
- » Man minimiert

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N ||f_{\theta}(x_i) - y_i||^2 + \mathcal{R}(y_i)$$



- » Bei Regularisierung durch Early Stopping wird Varianz dadurch reduziert, dass die Optimierung der Standard-Lossfunktion frühzeitig abgebrochen wird.
- » Man findet also nicht den eigentlichen Minimierer  $\theta^*$
- » Validierungsdaten können als Entscheidungsgrundlage für den Zeitpunkt des Abbruchs dienen.
- » Sehr weit verbreitet und oft irgendwie unerkannt.

- » Bei Regularisierung durch Design wird schon die Modellklasse stark eingeschränkt.
- » Die Modellklasse wird so eingeschränkt, dass sie nicht die Komplexität hat auf Trainingsdaten überanzupassen.
- » Beispiel: Deep Image Prior, Equivariant Networks, Input Convex Networks, ...
- » Das Vorwissen ist in der gewählten Modellklasse

# Beispiele



**FH KUFSTEIN TIROL**  
University of Applied Sciences

# $L^2$ Regularisierung

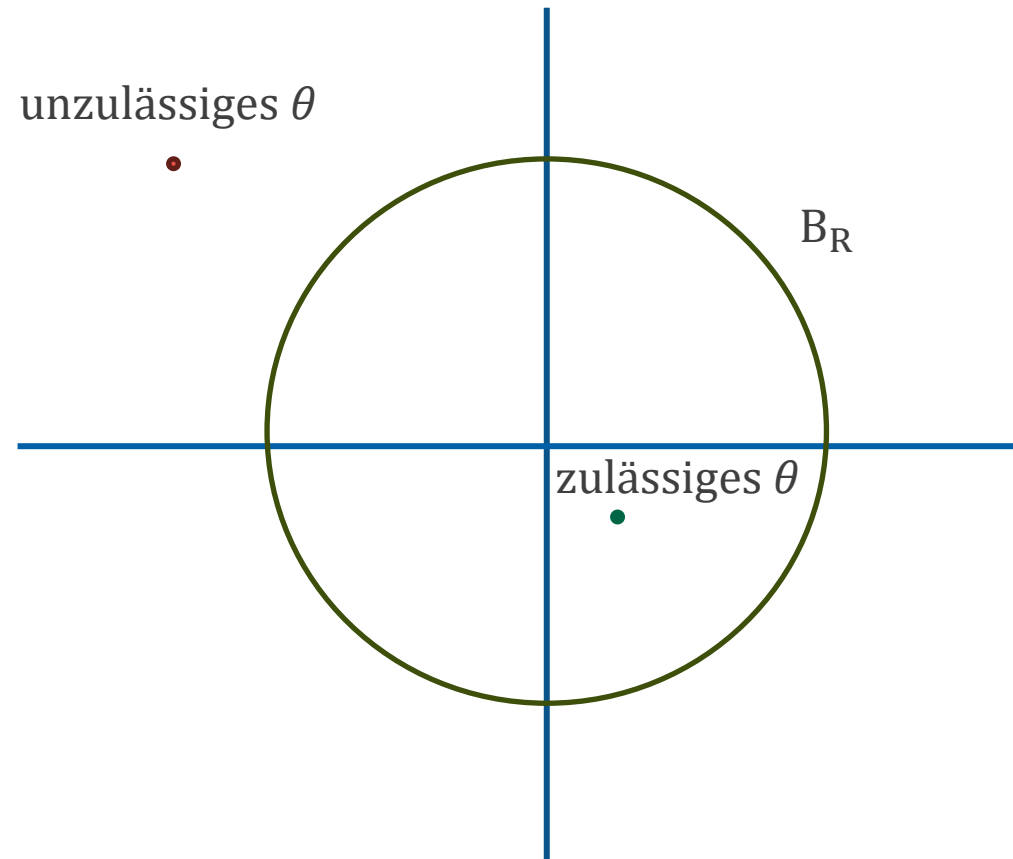
- » Bei der  $L^2$  –Regularisierung wird die quadratische euklidische Norm der Parameter eingeschränkt

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

mit  $\theta \in B_R$  wobei  $B_R = \{\theta : \|\theta\| \leq R\}$ .

- » Die  $L^2$  –Regularisierung hat viele verschiedene Namen:
- Tikhonov-Regularisierung
  - Ridge Regression
  - Weight Decay
  - Quadratische Regularisierung
- » Effekt: Stetigkeit

# $L^2$ Regularisierung geometrisch



- » Es gibt mehrere Möglichkeiten, wie man diese Zusatzbedingung implementieren kann.
- » Beispiele:
  - Minimierung mit Strafterm
  - Projected Gradient Descent
  - Proximal Methods
- » Die möglicherweise am weitesten verbreitete Methode ist die variationelle Regularisierung
- » Mehr dazu im Optimierungsteil dieser Lehrveranstaltung

- » Bei der variationellen  $L^2$  –Regularisierung wird eine große quadratische euklidische Norm bestraft

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N ||f_{\theta}(x_i) - y_i||^2 + \lambda ||\theta||^2$$

mit Regularisierungsparameter  $\lambda$ .

- » Die Wahl des Regularisierungsparameters ist schwierig:
- $\lambda = 0$  keine Regularisierung
  - $\lambda \rightarrow \infty$  „nur Regularisierung“ und  $\theta = 0$

- » Jede andere Norm kann als Regularisierungsfunktional verwendet werden
- » Das Regularisierungsfunktional kann eine Transformation der Parameter enthalten

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|T(\theta)\|^2$$



- » Bei der  $L^0$  –Regularisierung ist die Restriktion, dass eine gewisse Anzahl von Parametern Null sein müssen

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

mit  $\theta \in B_R^0$  wobei  $B_R^0 = \{\theta : \|\theta\|_0 \leq R\} = \{\theta : \#\{\theta_i \neq 0\} \leq s\}$ .

- » Optimierung ist nicht effizient möglich

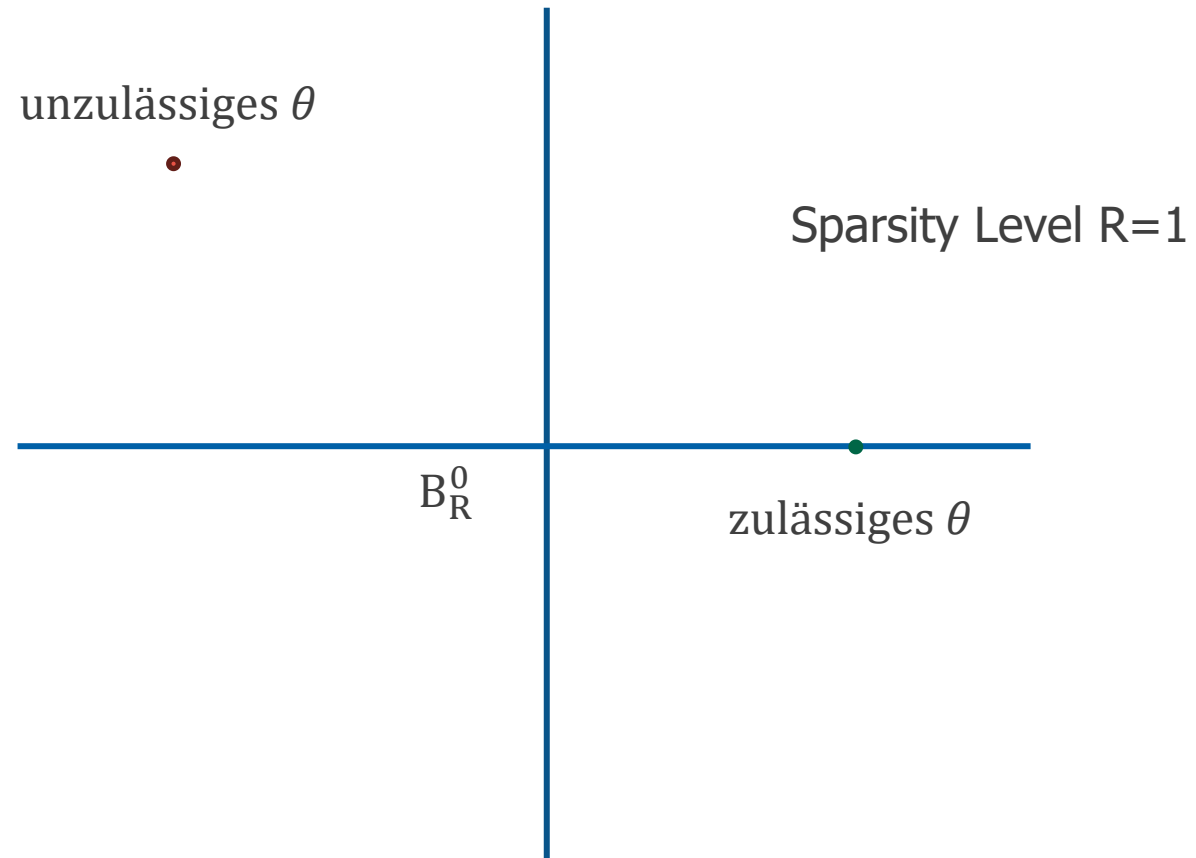
- » Bei der variationellen  $L^0$  –Regularisierung wird

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|\theta\|_0^2$$

mit Regularisierungsparameter  $\lambda$  minimiert.

- »  $\|\cdot\|_0$  ist keine echte Norm
- »  $\|\cdot\|_0$  ist nicht differenzierbar
- » Exakte Optimierung im Allgemeinen nur durch ausprobieren möglich

# Sparsity geometrisch



- » Um das Problem effizient lösen zu können wird das Regularisierungsfunktional abgeändert in der Hoffnung (theoretische Resultate) dass Sparsity Eigenschaften erhalten bleiben:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \|\theta\|_1^2$$

mit Regularisierungsparameter  $\lambda$  minimiert.

- » Wieder verschiedene Namen:
- Lasso
  - Basis Pursuit
  - $L^1$  –Regularisierung
- » Anderer Ansatz: greedy Algorithmen

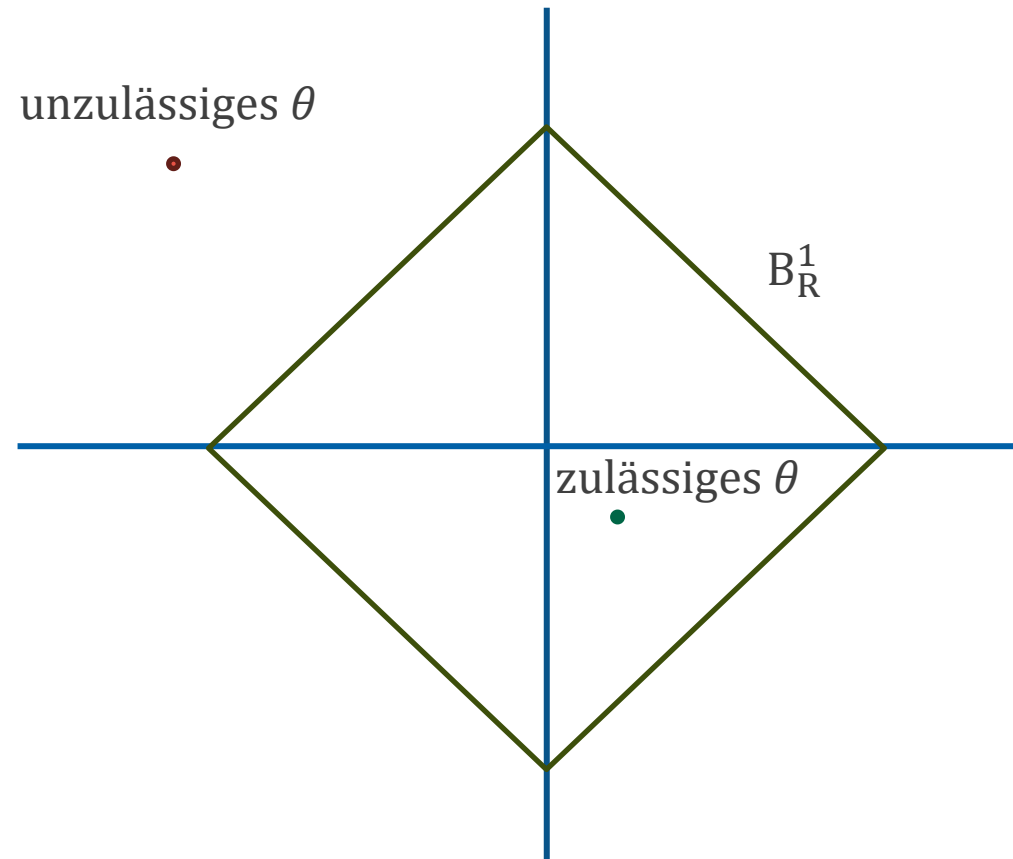
# $L^1$ Regularisierung

» Bei der  $L^1$  –Regularisierung wird die 1-Norm der Parameter eingeschränkt

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

mit  $\theta \in B_R^1$  wobei  $B_R^1 = \{\theta : \|\theta\|_1 \leq R\}$ .

# $L^1$ Regularisierung geometrisch



- » Es ist auch möglich mehrere Regularisierungsfunktionale zu verwenden

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N ||f_{\theta}(x_i) - y_i||^2 + \lambda_1 \mathcal{R}_1(\theta) + \lambda_2 \mathcal{R}_2(\theta)$$

- » Beispiel: Kombination aus  $L^1$  und  $L^2$  Regularisierung wird elastic net Regularisierung genannt.

- » Eine andere Möglichkeit ist die Regularisierung des Outputs
- » Besonders hilfreich, wenn sich Eigenschaften der Output-Signalklasse gut beschreiben lassen (z.B. Bilder)

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2 + \lambda \mathcal{R}(f_{\theta}(x_i))$$

- » Lässt sich auch mit den anderen Regularisierungsmethoden kombinieren



## Beispiele:

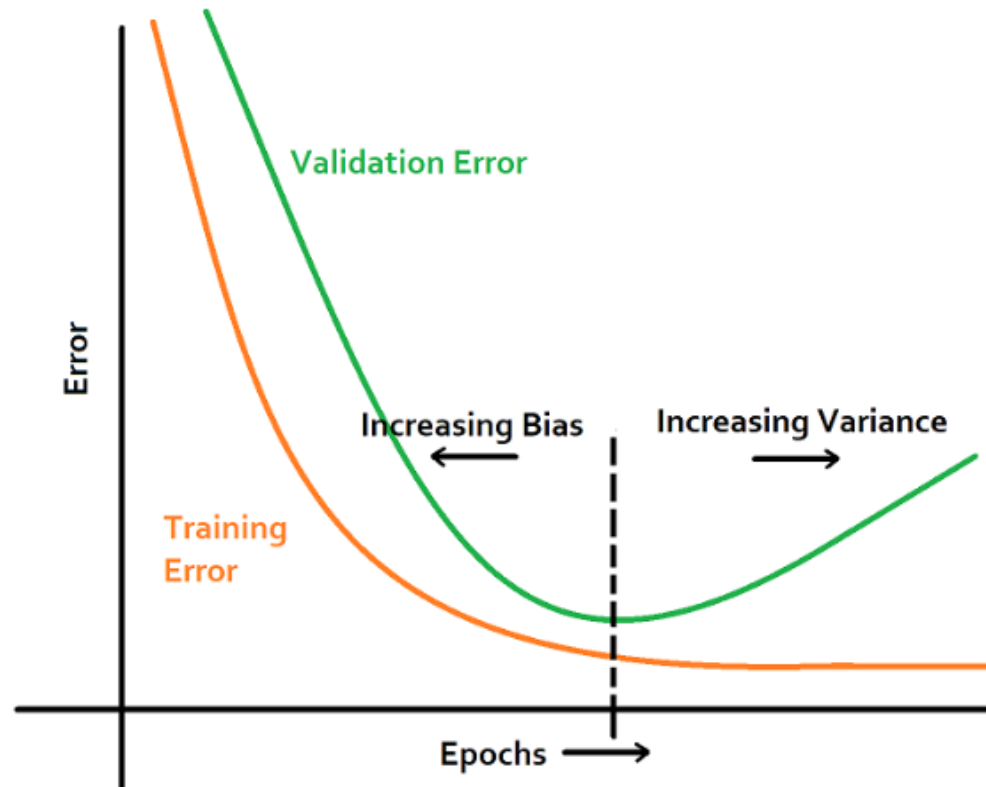
- » Transformationen des Outputs (z.B. Differentialoperatoren)

$$\mathcal{R}(x) = ||T(x)||^2$$

- » Shape Priors: Die Vorhersage soll bestimmte Formen bevorzugen
- » Topologie Priors: Vorwissen über Zusammenhangskomponenten, Löcher, ...
- » Gelernte Priors: Vortrainierte neuronale Netze als Regularisierungsfunktional
- » Viele weitere Möglichkeiten

# Early Stopping

- » Falls die Parameter mit einem iterativen Verfahren optimiert werden ist eine Möglichkeit der Regularisierung das early stopping.



- » Wenn die Performance auf dem Validierungsdatensatz nachlässt wird der Optimierungsprozess gestoppt
- » Das Model hat zu diesem Zeitpunkt eine kleine Varianz und man hat Evidenz dafür, dass es gut generalisiert
- » Weiteres Training würde zu overfitting führen.
- » Für ein simples lineares Modell kann man zeigen, dass early stopping des Gradientenabstiegsverfahren äquivalent zu  $L^2$  –Regularisierung ist

- » Unter Regularisierung durch Design verstehe ich die Wahl einer Modellklasse die nicht dazu in der Lage ist auf die Daten überanzupassen.
- » Ein Beispiel ist der Deep Image Prior (CNNs generell)
- » Auch adaptive Lösungen sind denkbar

- » Beim Dropout werden Bestandteile des Modells zufällig weggenommen.
- » Kann als Ensemble von mehreren Modellen (die viele Parameter teilen) gesehen werden
- » Das weglassen der Bestandteile wird nur während der Optimierung gemacht
  - Bei der Auswertung des optimierten Modells werden alle Bestandteile verwendet (z.B. für ein neuronales Netz in pytorch: `model.train -> model.test`)
  - In speziellen Fällen ist Dropout in der Inference möglich um eine Konfidenz der Vorhersage zu schätzen

- » Praxisempfehlungen:
  - Fully-connected: Wahrscheinlichkeit 0.2-0.5
  - Convolutional: eher kleiner 0.1-0.3
  - Kombination mit  $L^2$  –Regularisierung
- » Hyperparameter:
  - Höhere Dropout Rate: stärkere Regularisierung
  - Eventuell kleinere Werte in tieferen Schichten
  - Bei kleinen Datensätzen nicht zu hoch
- » Fallstricke
  - Zu hohe Rate → Lernverlangsamung, hoher Bias, degradierte Repräsentationen
  - In kleinen Netzen kann  $L^2$ /Design-Regularisierung effektiver sein als starkes Dropout

- » Idee:
  - Reduziert Überkonfidenz der Modelle
  - Ersetzt harte One-Hot Labels durch „weiche“ Zielverteilungen
  
- » Mehrere Varianten existieren:
  - Für  $\varepsilon \in [0,1]$ :  $y_c = (1 - \varepsilon) \cdot y + \varepsilon \cdot (1 - y)$
  
- » Das weglassen der Bestandteile wird nur während der Optimierung gemacht
  - Bei der Auswertung des optimierten Modells werden alle Bestandteile verwendet (z.B. für ein neuronales Netz in pytorch: `model.train -> model.test`)
  - In speziellen Fällen ist Dropout in der Inference möglich um eine Konfidenz der Vorhersage zu schätzen

- » Wirkung:
  - Weniger überkonfidente Vorhersagen
  - Stabilisiert Training bei Label-Noise
  - Leicht geringere Spitzen-Accuracy
  
- » Praxis:
  - Typisches  $\varepsilon$  klein
  - Nur auf Trainingslabels, nicht auf Val/Test
  - Nicht mit starker Klassenimbalance verwechseln
  
- » Fallstricke:
  - Zu großes  $\varepsilon$  führt zu untertrainierten glatten Entscheidungen
  - Bei stark unbalancierten Klassen nicht gut



# Notebook



**FH KUFSTEIN TIROL**  
University of Applied Sciences

- » Wir betrachten Wahrscheinlichkeiten
- » Für einen Datensatz  $D = \{(x_i, y_i)\}$  ist die Likelihood (Wahrscheinlichkeit, dass die Daten durch ein bestimmtes Modell entstanden sind)

$$p(\{(x_i, y_i)\}|\theta).$$

- » Wir wollen die Posterior Wahrscheinlichkeit (Wahrscheinlichkeit eines Modells wenn die Daten gegeben sind) maximieren

$$p(\theta|\{(x_i, y_i)\})$$

- » Durch den Satz von Bayes erhalten wir

$$p(\theta | \{(x_i, y_i)\}) \propto \underbrace{p(\{(x_i, y_i)\} | \theta)}_{\text{Likelihood}} \cdot \underbrace{p(\theta)}_{\text{Model Prior}}$$

- » Ein Output Prior wurde in dieser Modellierung nicht berücksichtigt.
- » Anstatt diese Wahrscheinlichkeit zu maximieren kann man äquivalenterweise auch den Logarithmus davon maximieren oder mit negativem Vorzeichen minimieren

$$-\log p(\theta | \{(x_i, y_i)\}) = -\log p(\{(x_i, y_i)\} | \theta) - \log p(\theta)$$

## » Die Standardwahl

$$p(\{(x_i, y_i)\}|\theta) = \prod_{i=1}^N p((x_i, y_i)|\theta) = \prod_{i=1}^N \exp(-\|f_{\theta}(x_i) - y_i\|^2)$$

liefert

$$-\log p(\{(x_i, y_i)\}|\theta) = \sum_{i=1}^N \|f_{\theta}(x_i) - y_i\|^2$$

wobei hier die Varianzen weggelassen wurden und die Gleichheiten nur bis auf Konstanten stimmen

» Standardwahlen für den Prior sind

$$p(\theta) = \exp(-||\theta||^2)$$

und

$$p(\theta) = \exp(-||\theta||_1)$$

Wodurch sich die  $L^2$  und  $L^1$  Regularisierung ergeben. (Varianzen und normierungen wurden wieder weggelassen)

# Entscheidungsschema: Wann benutze ich welche Regularisierung?

- » **Ziel:** Passende Regularisierung aus Problemcharakter, Datenlage und Vorwissen ableiten.
- » Kein allgemeines Schema sondern nur Vorschläge.

# Entscheidungsschema

| Szenario                     | Empfehlung                                             | Hinweise                                        |
|------------------------------|--------------------------------------------------------|-------------------------------------------------|
| Wenig Daten – viele Features | L2/L1, Early Stopping, einfache Architektur            | Standardisieren, Cross-Validation für $\lambda$ |
| Strukturierte Outputs        | TV/Laplace                                             | Output Regularisierung ist oft hilfreich        |
| Starkes Vorwissen            | Regularisierung durch Design                           | Kapazität begrenzen, Induktive Bias nutzen      |
| Optimierung instabil         | Gradient-Clipping, Max-Norm, Non-Negativity bei Bedarf | Kleinere Learning Rate                          |

- » Normierung:
  - Output Regularisierer durch Anzahl der Ausgabekomponenten (z.B. Pixel) dividieren (->  $\lambda$  wird skaleninvariant)
  - Bei Summen über Samples: Mittelwert statt Summe
- » Suche:
  - Logarithmisches statt lineares Gitter
  - Coarse-to-fine: Zuerst Grobes Raster
  - Budgetiert und robust: Wenige Epochen für Grobsuche
- » Stabilität:
  - Stabile Optimierer (später)
  - Early Stopping



# Hyperparameter Richtwerte

- »  $L^2$ :  $1e - 6 \dots 1e - 2$
- »  $L^1$ :  $1e - 6 \dots 1e - 3$
- » Max-Norm:  $1 \dots 3$  (pro Layer)
- » Dropout-Rate:  $0.1 \dots 0.5$
- » Patience für Early Stopping: 5-10
- » Lernraten-Schedule oft hilfreich

# Hyperparameter Auswahlprozess

- » Split: Train/Val/Test sauber trennen; stratifizieren bei Klassifikation
- » Pro Regularisierer separat grob sweepen ( $\lambda$  auf Log-Gitter)
- » Kombinationen nur in Top-2–3-Bereichen verfeinern
- » Ergebnisse mitteln über 3–5 Seeds bei kleinem N
- » Finales Modell auf Train+Val neu trainieren; Test-Set nur einmalig reporten

# Diagnose Unter vs. Über-Regularisierung

- » Unter-Regularisierung (zu kleine  $\lambda$ ):
  - Hohe Varianz, große Lücke Train vs. Val, instabile Grenzen
  - Maßnahme:  $\lambda$  erhöhen; Kapazität senken; Early Stopping aggressiver
- » Über-Regularisierung (zu große  $\lambda$ ):
  - Hoher Bias, beides schlecht (Train und Val), verschwommene/zu glatte Outputs
  - Maßnahme:  $\lambda$  reduzieren; Kapazität leicht erhöhen; Datenaugmentation prüfen
- » Output-Regularisierung spezifisch:
  - Zu viel TV: Überglättete, verkürzte Strukturen; zu wenig feine Details
  - Zu wenig TV: Zacken/Rauschen; „flatternde“ Grenzen

- » Standard-Baseline:
  - Start mit kleinem  $L^2$  ( $1e-4$ ), Early Stopping, kein Dropout
  - Danach: Output-Regularisierung hinzufügen falls Outputs strukturiert sind
- » Skalierungs-Check:
  - Input-Standardisierung; Loss-Skalen prüfen (Daten-Term vs. Regularisierungsterm)
  - Regularisierungsterm per Batch mitteln, nicht summieren
- » Dokumentation:
  - Alle  $\lambda$  und Seeds loggen; Trainingskurven speichern; Reproduzierbare Splits

# Cross Validation



**FH KUFSTEIN TIROL**  
University of Applied Sciences

- » Unter Cross-Validation versteht man Techniken um herauszufinden wie ein machine learning Modell auf ungesehen Daten performt
- » Cross-Validation kann verwendet werden um
  - Das Modell zu evaluieren
  - Das Modell aus verschiedenen Modellen auszuwählen
  - Für Parameter Tuning
  - Um overfitting zu vermeiden

## » Wie funktioniert:

- Der Datensatz wird in mehrere Teile geteilt
- Das Modell wird auf einigen trainiert und auf den anderen getestet
- Das wird einige Male wiederholt indem verschiedene Teile ausgewählt werden
- Die finale Performance wird als Durchschnitt der einzelnen Validierungsschritte ermittelt

- » **Hold-Out Cross-Validation:** Der Datensatz wird in zwei Teile geteilt, typischerweise 70-80% um das Modell zu fitten und 20-30% um es zu testen
- » **K-Fold Cross Validation:** Datensatz wird in K gleiche Teile geteilt. Das Modell auf  $K - 1$  Folds trainiert und auf dem Übrigen getestet. Dieser Prozess wird K mal wiederholt, jedes mal mit unterschiedlichem Test set. Die finale Performance ist der Durchschnitt der K Durchgänge.
- » **Stratified Cross Validation:** Ist eine Variation der K-Fold Cross Validation bei der, der Prozentsatz der Klassen in den Splits gleich ist.
- » **Leave One Out Cross Validation:** Jeder Datenpunkt wird einmal als Test set verwendet und auf den übrigen trainiert. Das Modell wird N mal trainiert.